

Problem Description

As students, we face many assignments, tasks, and exams daily. Hence, there are many cases where students are confused about where to start and what to focus on, resulting in deadlines slipping through the cracks without a proper organization system. This not only leads to extra loads of stress, but it also leads to lower grades and overall performance and productivity.

In order to fix that, a program was designed to keep track of students' various tasks. Helping them keep track of what tasks they have to do and what they have completed. Through a simple menu set up, students can add new tasks with validation, view current tasks, mark tasks as completed, and delete tasks. This helps them be better organized and use their time more effectively.

Program Design

The program starts by telling the user to upload a text file named 'tasks.txt'. The code then starts processing by loading lines from the file and formatting it. The 6 Key functions are defined next, explained in the next sections. The program output starts with the first print option, welcoming the user and asking to upload a text file. The code next calls the menu function to display the operations to the user and assigns the selection variable. A while loop is added where the user is prompted to select which menu operation they would like to use. The user can select between adding, viewing, removing, or marking a task as complete. The loop also uses a try and except block, which identifies when the wrong data type (not an integer in this case) is entered and tells the user to try again. The program also prompts the user to enter their selection again if they enter an out of range integer.

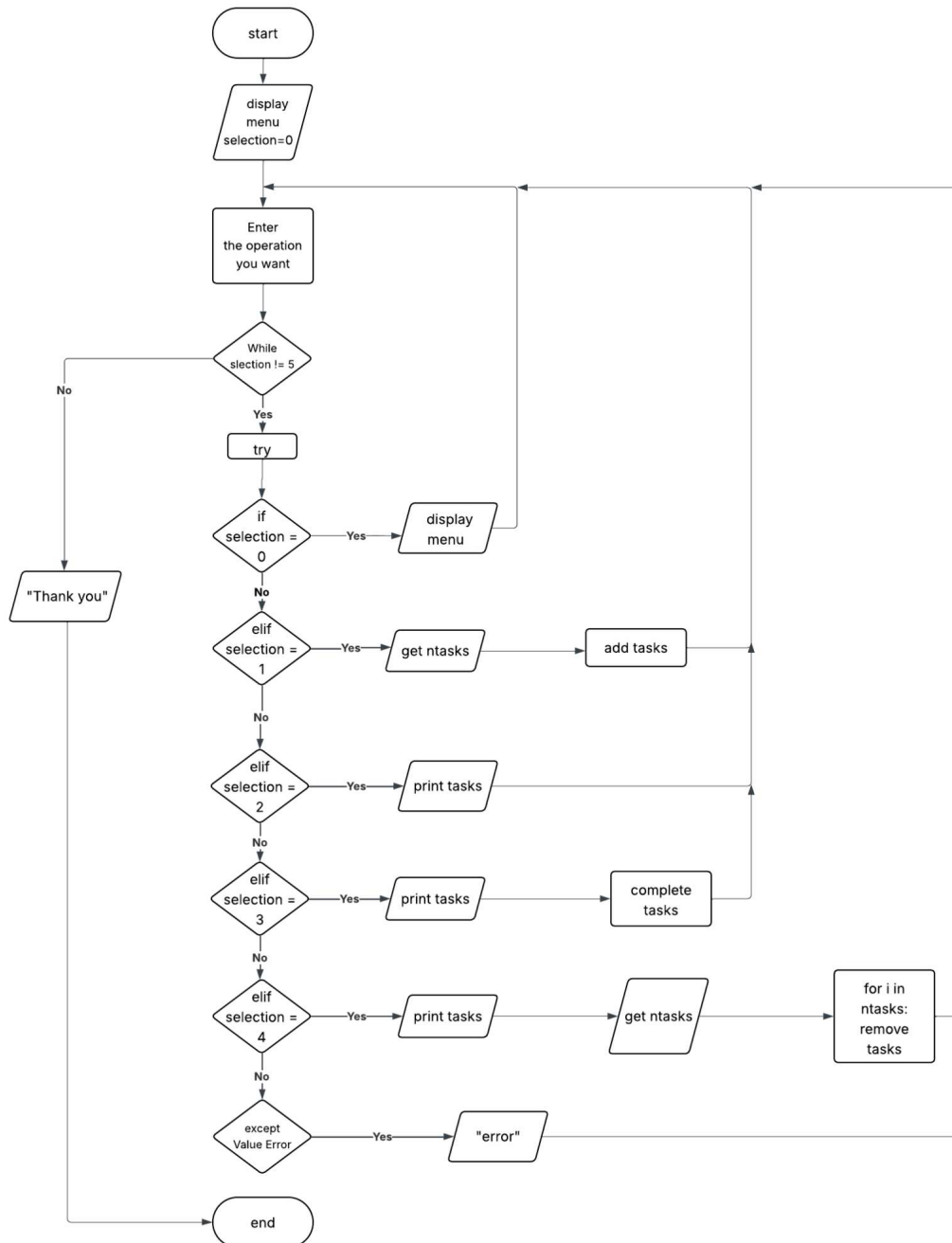
Key Functions

The program uses 6 user defined functions:

1. Saving to File function which is recalled after every change to the to-do list to update the file version as well
2. Menu Function to display all the operations a user can do with their to-do list
3. Adding task function which prompts the user to add as many tasks as they like, simultaneously updates the list as well by calling the save_tasks function
4. Print tasks function shows all the tasks the user has and their status. It uses the enumerate function to add a counter to the list

5. Complete task function which prompts the user to identify the task they completed by the task numbers (which is then decreased by 1 to match the list indices) after showing them their tasks. The function adds a checkmark icon next to the chosen task.
6. Remove task function also works similar to the complete task function, but it deletes the selected task from the list instead.

Flowchart



Program Output Screenshots

Main Menu:

```
... Please upload a text file named 'tasks.txt' to ensure smooth running.
Download it at the end to keep a record of today's tasks in your device!

Welcome to your PMU Coursework to-do list!

Menu:
1. Add new task.
2. Show all tasks.
3. Completed a task.
4. Remove a task.
5. Exit.

Enter the operation you want to procede with (Enter 0 to view the menu again)

```

Adding a task:

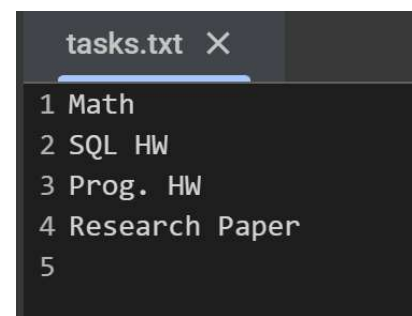
```
Enter the operation you want to procede with (Enter 0 to view the menu again)
1
How many tasks would you like to add? 3
Enter task: SQL HW
Added!

Enter task: Prog. HW
Added!

Enter task: Research Paper
Added!

Enter the operation you want to procede with (Enter 0 to view the menu again)

```

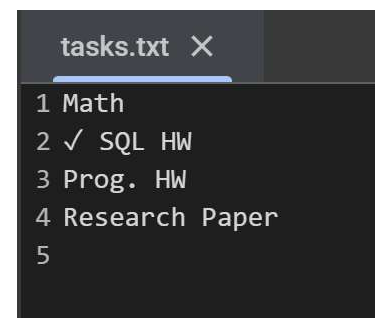


Completing a task:

```
Enter the operation you want to procede with (Enter 0 to view the menu again)
3
Enter the task number you have complete: 2
Congratulations on completing your task!

Your tasks:
1. Math
2. ✓ SQL HW
3. Prog. HW
4. Research Paper
Enter the operation you want to procede with (Enter 0 to view the menu again)

```



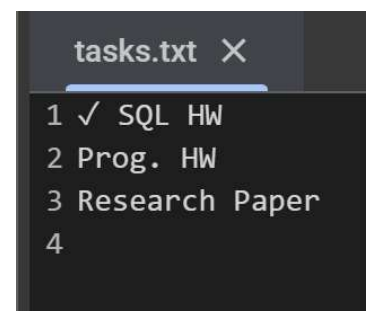
Removing a task:

```
Enter the operation you want to procede with (Enter 0 to view the menu again)
4

Your tasks:
1. Math
2. ✓ SQL HW
3. Prog. HW
4. Research Paper
How many tasks would you like to remove? 1
Enter the task number you want to remove 1
Task deleted.

Your tasks:
1. ✓ SQL HW
2. Prog. HW
3. Research Paper
Enter the operation you want to procede with (Enter 0 to view the menu again)

```



Viewing all tasks:

```
Enter the operation you want to procede with (Enter 0 to view the menu again)
2

Your tasks:
1. ✓ SQL HW
2. Prog. HW
3. Research Paper
Enter the operation you want to procede with (Enter 0 to view the menu again)

```

Exit:

```
Enter the operation you want to procede with (Enter 0 to view the menu again)
5
Thank you for using our program.
```

Errors:

```
Enter the operation you want to procede with (Enter 0 to view the menu again)
a
Error. Enter the correct number.

Enter the operation you want to procede with (Enter 0 to view the menu again)
20
Try again. Enter the correct number.

Enter the operation you want to procede with (Enter 0 to view the menu again)

```

Lessons Learned

Throughout the design of this program, there were a few obstacles that were presented. Firstly, as the program is more complex than our previous coding experiences, a lot of pre-planning had to be done in order to adequately organise the code logically. Hence, we decided to make sure to break the program into small pieces and add plenty of notes to improve its readability. That, in turn, helped us when we faced errors, as we were able to solve them more quickly. Moreover, by practically using all the concepts we have learned, we were able to see the different errors that can arise. For example, while trying to implement file handling, we faced many errors until we understood we had to upload a file first and then start. Furthermore there are concepts we didn't learn in the course but discovered along the way that made the code run more smoothly like the try-except which we learned while looking for a solution for an error that kept repeating. This piece of code helps in error handling as it allows the program to "try" a block of code that might cause an error and catch it to run the "except" block, which in our case prints an error message. Other types of

obstacles we faced were the integration of the different ideas that the group members had as we decided to split the work and the functions in the beginning. However, that led to some differences in the way each of us wrote their part. Ultimately in the end, we organized and unified our code to improve the readability of our program. Through this assignment, we practically implemented the concepts that we learned in class and were able to see where we are lacking and what errors could arise from coding, in addition to finding new concepts in our journey to make a functional and useful program.